AlquilaTec S.A.S.

Definition of Ready (DoR) y Definition of Done (DoD)

Proyecto AlquilaTec S.A.S. - Plataforma Hardware as a Service (HaaS)

Documento de gestión ágil para historias de usuario, issues y Pull Requests

Proyecto	AlquilaTec S.A.S.
Producto	Plataforma web de Hardware as a Service (HaaS)
Equipo de desarrollo	Los Tipitos
Metodología	Scrum
Stack técnico	Laravel, React, MariaDB, GitHub, Docker, CI/CD
Versión	1.0

Introducción

Este documento define la Definition of Ready (DoR) y la Definition of Done (DoD) para el proyecto AlquilaTec S.A.S. Su objetivo es establecer reglas claras para saber cuándo una historia de usuario, tarea técnica o issue está lista para iniciar y cuándo puede considerarse realmente terminada.

Las definiciones están adaptadas al contexto del proyecto: una plataforma web de alquiler de activos tecnológicos bajo el modelo Hardware as a Service, con roles de arrendador, arrendatario y administrador, desarrollada con Laravel, React, MariaDB, GitHub, Docker y prácticas de integración y despliegue continuo.

Aplicación del documento

- Debe usarse durante el refinamiento del backlog, planificación de sprint, revisión de Pull Requests y cierre de tareas.
- Aplica a historias de usuario, tareas técnicas, correcciones de errores, ajustes visuales, cambios de base de datos y entregables de sprint.
- Sirve como criterio de control para evitar iniciar tareas ambiguas o cerrar trabajo incompleto.

1. Definition of Ready (DoR)

La Definition of Ready define las condiciones mínimas que debe cumplir una historia de usuario, tarea técnica o issue antes de iniciar su desarrollo dentro de un sprint.

En AlquilaTec, un ítem se considera Ready cuando el equipo puede comprenderlo, estimarlo, implementarlo y validarlo sin depender de suposiciones importantes.

1.1 Criterios obligatorios de entrada

A. Contexto de negocio y valor esperado

- El ítem debe explicar qué problema, necesidad u oportunidad resuelve.
- Debe identificar el actor impactado: arrendador, arrendatario, administrador o sistema.
- Debe indicar qué parte del flujo de alquiler se ve afectada.
- Debe explicar por qué es importante para el MVP o para el producto completo.

B. Alcance funcional

- Debe describir qué debe hacer la funcionalidad.
- Debe aclarar qué queda fuera del alcance.
- Debe identificar datos, reglas de negocio, pantallas, formularios o flujos afectados.
- Para el MVP debe priorizarse el flujo principal: registro, roles, publicación de productos, catálogo, disponibilidad, solicitud de reserva, aprobación o rechazo y aceptación de políticas.

C. Criterios de aceptación verificables

- Deben ser claros, medibles y comprobables mediante pruebas manuales o automatizadas.
- Deben relacionarse directamente con el comportamiento esperado.
- Deben incluir casos felices, validaciones y casos de error relevantes.
- No debe iniciar una tarea cuyo resultado no pueda validarse objetivamente.

D. Preparación de UX/UI

- Cuando el ítem afecte la interfaz, debe existir mockup, wireframe, captura de referencia, diseño o descripción visual suficiente.
- Deben definirse los estados de interfaz: inicial, carga, vacío, error y éxito.
- Debe aclararse el comportamiento esperado en escritorio y móvil básico cuando aplique.
- Deben definirse textos principales, mensajes de error y mensajes de éxito cuando sean relevantes.

E. Preparación técnica

- Debe identificarse si el cambio afecta frontend React, backend Laravel, base de datos MariaDB, autenticación, API, storage, Docker o despliegue.
- Deben identificarse componentes, rutas, controladores, modelos, migraciones, endpoints o servicios posiblemente afectados.
- Debe saberse si se requieren nuevas variables de entorno, seeders, políticas, validaciones o configuraciones adicionales.
- Debe existir una ruta técnica razonable para implementar el cambio.

F. Datos, entidades y reglas de negocio

- Deben identificarse las entidades involucradas: usuarios, roles, productos, reservas, pagos, depósitos, evidencias, calificaciones, reclamos o transacciones.
- Deben definirse campos obligatorios, relaciones, estados posibles, restricciones y validaciones.
- Deben respetarse las reglas de negocio del alquiler, como disponibilidad, depósito, aceptación de políticas y trazabilidad.
- Debe indicarse si la tarea afecta datos sensibles o documentos de identidad.

G. Seguridad y permisos

- Debe definirse qué rol puede ejecutar la acción y qué rol no puede hacerlo.
- Debe indicarse si la acción requiere autenticación.
- Debe aclararse si se manejan imágenes, documentos, evidencias o datos sensibles.
- Deben considerarse reglas de autorización para acciones administrativas o privadas.

H. Dependencias, bloqueos y riesgos

- Deben listarse dependencias previas, bloqueos y supuestos relevantes.
- Debe indicarse si la tarea depende de otro endpoint, diseño, migración, componente o decisión del equipo.
- Deben registrarse riesgos técnicos o funcionales conocidos.
- Si el riesgo es medio o alto, debe existir una estrategia de mitigación.

I. Planificación y responsable

- El ítem debe tener responsable asignado.
- Debe tener prioridad definida y sprint objetivo.
- Debe tener un tamaño abordable para la iteración.
- Si la tarea es demasiado grande, debe dividirse antes de entrar al sprint.

J. Trazabilidad en GitHub

- Toda tarea debe existir como issue o estar asociada a una issue existente.
- Debe estar vinculada al tablero, sprint o milestone correspondiente.
- Debe tener labels, responsable y relación con la historia de usuario cuando aplique.
- La evidencia esperada al cierre debe estar definida desde el inicio.

1.2 Regla operativa del DoR

Una historia, tarea o issue solo puede moverse a Ready cuando cumple estas condiciones:

- El objetivo está claro.
- El actor impactado está identificado.
- El alcance y el fuera de alcance están definidos.
- Existen criterios de aceptación verificables.
- El equipo entiende cómo implementarlo técnicamente.
- No existen bloqueos que impidan iniciar.
- La tarea puede completarse dentro del sprint.

- Está registrada y trazable en GitHub.

Si alguno de estos puntos no se cumple, el ítem debe permanecer en Backlog, Refinement o To Do, pero no debe iniciar desarrollo.

1.3 Checklist operativo de DoR para GitHub Issues

No.	Verificación obligatoria	OK
1	El objetivo del ítem está descrito con claridad.	[ ]
2	El actor impactado está identificado: arrendador, arrendatario, administrador o sistema.	[ ]
3	El valor esperado para el negocio o el MVP está definido.	[ ]
4	El alcance funcional está descrito.	[ ]
5	El fuera de alcance está explícito.	[ ]
6	Existen criterios de aceptación verificables.	[ ]
7	Se definieron validaciones y casos de error.	[ ]
8	Existe referencia visual o lineamiento UX/UI si aplica.	[ ]
9	Se identificaron los estados de interfaz: carga, vacío, error y éxito si aplica.	[ ]
10	Se identificó el impacto técnico: frontend, backend, base de datos, auth, storage o despliegue.	[ ]
11	Se identificaron entidades, tablas, endpoints o componentes afectados.	[ ]
12	Se definieron permisos y restricciones por rol si aplica.	[ ]
13	Se identificaron dependencias, bloqueos y riesgos.	[ ]
14	La tarea tiene responsable asignado.	[ ]
15	La tarea tiene prioridad y sprint objetivo.	[ ]
16	La tarea tiene tamaño abordable para el sprint.	[ ]
17	La evidencia esperada al cierre está definida.	[ ]
18	La issue quedó vinculada al tablero, sprint o milestone correspondiente.	[ ]

1.4 Criterio de control del DoR

Una issue no debe moverse a In Progress si no supera este checklist. Las excepciones deben quedar registradas en la misma issue y aprobadas por el responsable del proyecto o Scrum Master.

2. Definition of Done (DoD)

La Definition of Done define las condiciones que debe cumplir una historia, tarea, issue o Pull Request para considerarse realmente terminada.

En AlquilaTec, una tarea no está terminada solo porque funciona en el computador de un integrante. Está terminada cuando cumple los criterios funcionales, técnicos, visuales, de seguridad, revisión, pruebas y documentación necesarios.

2.1 Criterios obligatorios de salida

A. Cumplimiento funcional

- El cambio debe cumplir completamente los criterios de aceptación definidos en la issue.
- El comportamiento implementado debe coincidir con lo solicitado.
- El flujo debe funcionar de inicio a fin cuando aplique.
- No deben entregarse funcionalidades incompletas como si estuvieran terminadas.
- Las desviaciones y pendientes deben quedar documentados.

B. Calidad de código

- El código debe ser claro, mantenible y coherente con la arquitectura del proyecto.
- Deben usarse nombres comprensibles y estructuras consistentes.
- Debe respetarse la separación entre frontend, backend y base de datos.
- No deben quedar logs innecesarios, código muerto, comentarios obsoletos o soluciones temporales ocultas.

C. Validación técnica

- El backend Laravel debe ejecutar sin errores si fue afectado.
- El frontend React debe compilar sin errores si fue afectado.
- Las dependencias deben estar instaladas correctamente.
- El flujo debe haber sido probado localmente o en ambiente de pruebas.
- El cambio no debe romper funcionalidades existentes.

D. Base de datos

- Todo cambio de esquema debe tener migración generada y versionada.
- Las migraciones deben ejecutarse correctamente.
- Deben respetarse relaciones, llaves foráneas y restricciones.
- No deben hacerse cambios manuales en base de datos como sustituto de migraciones.
- Si se requieren datos iniciales, deben usarse seeders cuando aplique.

E. API y contratos frontend-backend

- Los endpoints deben estar implementados y responder correctamente.
- El método HTTP, payload y estructura de respuesta deben ser consistentes.
- Los errores deben manejarse de forma controlada.
- El frontend debe consumir la API sin depender de datos quemados innecesarios.
- Las rutas deben respetar la estructura definida para la API cuando corresponda.

F. Seguridad, autenticación y permisos

- No deben subirse secretos, tokens ni credenciales al repositorio.
- Las rutas protegidas deben requerir autenticación.
- Cada usuario debe acceder únicamente a las funciones permitidas por su rol.
- No debe exponerse información sensible en respuestas, logs o errores.
- Las acciones administrativas deben permanecer restringidas a usuarios autorizados.

G. Calidad visual y experiencia de usuario

- La interfaz debe revisarse en escritorio y móvil básico cuando aplique.
- Deben existir estados de carga, vacío, error y éxito.
- Los formularios deben validar datos y mostrar mensajes claros.
- Los botones y acciones deben ser comprensibles.
- No deben quedar textos de prueba visibles.

H. Pruebas y evidencia

- Toda tarea debe tener evidencia de validación.
- La evidencia puede ser captura, video corto, checklist, pasos reproducibles, resultado de pruebas o link al Pull Request.
- Hasta que exista una suite madura de pruebas automatizadas, la evidencia manual es obligatoria.
- Si se agregan pruebas automáticas, deben estar alineadas con el comportamiento esperado.

I. Revisión mediante Pull Request

- Todo cambio debe integrarse mediante Pull Request.
- El PR debe estar asociado a una issue.

- Debe tener descripción clara del cambio y evidencia de validación.
- El autor debe realizar auto-revisión antes de solicitar revisión externa.
- Debe recibir revisión de al menos un compañero cuando sea posible y resolver los comentarios antes del merge.

J. Documentación

- Deben actualizarse README, variables de entorno, instrucciones de ejecución, documentación de endpoints o notas técnicas cuando el cambio lo requiera.
- La documentación debe reflejar el comportamiento real del sistema.
- Las evidencias del sprint deben quedar disponibles cuando correspondan.

K. Despliegue o integración

- El cambio debe integrarse en la rama correspondiente.
- Debe ejecutarse sin errores en ambiente local o de pruebas.
- Debe pasar validaciones de CI si están configuradas.
- Debe quedar listo para demostración cuando haga parte de una entrega de sprint o del MVP.
- No debe depender de configuraciones ocultas en el equipo de un desarrollador.

L. Cierre formal

- La tarea solo puede cerrarse cuando cumple todos los criterios de aceptación.
- Debe tener evidencia suficiente.
- No debe tener comentarios críticos abiertos.
- No debe dejar trabajo pendiente oculto.
- Lo pendiente debe convertirse en nuevas issues.

2.2 Estados recomendados antes de Done

Estado	Significado operativo
To Do	La tarea está lista para iniciar, pero aún no está en desarrollo.
In Progress	La tarea está siendo desarrollada.
Code Complete	El código fue implementado, pero falta validación, evidencia o revisión.
In Review	El Pull Request está en revisión técnica.
In QA	El cambio ya fue revisado, pero falta validación funcional o evidencia final.
Done	El cambio cumple completamente el DoD y no tiene pendientes ocultos.

2.3 Checklist operativo de DoD para Pull Requests

No.	Verificación obligatoria	OK
1	El cambio cumple todos los criterios de aceptación.	[ ]
2	El flujo funcional fue validado de extremo a extremo si aplica.	[ ]
3	El cambio no rompe funcionalidades existentes.	[ ]
4	El backend Laravel ejecuta correctamente si aplica.	[ ]
5	El frontend React compila correctamente si aplica.	[ ]
6	Se ejecutaron migraciones si hubo cambios de base de datos.	[ ]
7	Las migraciones quedaron versionadas.	[ ]
8	No se hicieron cambios manuales en base de datos como solución final.	[ ]

9	Los endpoints, payloads y respuestas fueron validados si aplica.	[ ]
10	Se validaron permisos, roles y autenticación si aplica.	[ ]
11	No se expusieron secretos, tokens ni credenciales.	[ ]
12	Se validaron estados de carga, vacío, error y éxito si aplica.	[ ]
13	La interfaz fue revisada en desktop y móvil básico si aplica.	[ ]
14	Los mensajes de error y éxito son claros para el usuario.	[ ]
15	Se adjuntó evidencia de validación.	[ ]
16	El Pull Request está enlazado a la issue correspondiente.	[ ]
17	El autor realizó auto-revisión del cambio.	[ ]
18	El PR recibió revisión de otro integrante cuando fue posible.	[ ]
19	Los comentarios de revisión fueron resueltos.	[ ]
20	La documentación necesaria fue actualizada.	[ ]
21	El cambio fue integrado en la rama correspondiente.	[ ]
22	No quedan pendientes ocultos; lo restante quedó trazado en nuevas issues.	[ ]

2.4 Criterio de control del DoD

Una tarea no debe moverse a Done si conserva criterios de aceptación incumplidos, validaciones pendientes, errores conocidos no documentados, falta de evidencia, comentarios críticos abiertos en el PR, problemas de permisos o seguridad, o pendientes ocultos dentro del mismo objetivo.

La frase "ya funciona" no es suficiente. Debe existir evidencia verificable.

3. Aplicación práctica en GitHub

3.1 Plantilla recomendada para Issues

```
## Objetivo

Describir qué se quiere lograr con esta tarea.


## Actor impactado

- [ ] Arrendador

- [ ] Arrendatario

- [ ] Administrador

- [ ] Sistema


## Historia de usuario relacionada

Ejemplo: HU14 - Como arrendatario, quiero solicitar el alquiler de un producto por un tiempo
determinado.


## Alcance

Qué incluye esta tarea.


## Fuera de alcance

Qué no incluye esta tarea.
```

Criterios de aceptación

- [ ] Criterio 1
- [ ] Criterio 2
- [ ] Criterio 3

Impacto técnico

- [ ] Frontend
- [ ] Backend
- [ ] Base de datos
- [ ] Auth / permisos
- [ ] Storage
- [ ] API
- [ ] DevOps / despliegue

Dependencias o bloqueos

Indicar si depende de otra tarea, endpoint, diseño, migración o decisión.

Evidencia esperada

Capturas, video, checklist, pruebas, link del PR o demo funcional.

3.2 Plantilla recomendada para Pull Requests

Resumen del cambio

Explicar brevemente qué se implementó.

Issue relacionada

Closes #___

Tipo de cambio

- [ ] Nueva funcionalidad
- [ ] Corrección de error
- [ ] Refactor
- [ ] Cambio de UI
- [ ] Cambio de base de datos
- [ ] Documentación
- [ ] Configuración / DevOps

Validaciones realizadas

- [ ] Probado localmente
- [ ] Flujo validado de extremo a extremo

```
- [ ] Migraciones ejecutadas si aplica
- [ ] Permisos revisados si aplica
- [ ] UI revisada en desktop y móvil si aplica
- [ ] No se expusieron secretos

## Evidencia
Agregar capturas, video corto o pasos reproducibles.

## Pendientes
Indicar si queda algo por fuera. Si existe, debe tener nueva issue.
```

4. Versión resumida para anexar al documento del proyecto

Definition of Ready resumida

Una tarea está lista para iniciar cuando tiene objetivo claro, actor identificado, valor esperado, alcance y fuera de alcance definidos, criterios de aceptación verificables, impacto técnico identificado, permisos claros si aplica, dependencias registradas, responsable asignado, tamaño abordable y trazabilidad en GitHub.

Definition of Done resumida

Una tarea está terminada cuando cumple todos sus criterios de aceptación, fue probada funcional y técnicamente, no rompe funcionalidades existentes, respeta roles y seguridad, tiene evidencia de validación, cuenta con Pull Request revisado, documentación actualizada si aplica y no deja pendientes ocultos.

5. Recomendación de uso

- Usar el checklist de DoR antes de mover una issue a In Progress.
- Usar el checklist de DoD antes de cerrar un Pull Request o mover una tarea a Done.
- Mantener las evidencias de validación dentro de GitHub para que cada sprint pueda auditarse.
- No cerrar tareas con pendientes ocultos; cualquier trabajo restante debe convertirse en nueva issue.
- Actualizar este documento si el proyecto cambia de arquitectura, metodología o alcance.